

MVP

Engenharia de Dados

Pipeline de Dados para Análise de Satisfação do Cliente e Desempenho Logístico em E-commerce (Olist)

Aluno: Pietro Santos Thompson da Cunha
GitHub: https://github.com/PietroThompson/MVP-Data_engineering

Sumário

Sumário	2
Estudo de Satisfação do Cliente e Desempenho Logístico em E-commerce	3
Contextualização e objetivos gerais.....	3
Detalhamento do projeto	3
Dados e ferramentas selecionadas	3
Coleta e carregamento de dados.....	4
Camada Bronze	4
Camada Silver.....	6
Camada Gold.....	8
Catálogo de dados.....	9
Análise de dados	16
Qualidade de dados	16
Solução do problema proposto.....	16
Considerações finais	25
Pontos de melhoria.....	25
Autoavaliação.....	25

Estudo de Satisfação do Cliente e Desempenho Logístico em E-commerce

Contextualização e objetivos gerais

O mercado de e-commerce brasileiro é altamente competitivo, e a experiência de entrega é um dos principais fatores que influenciam a satisfação e a retenção de clientes em um marketplace. Entender quais fatores operacionais e logísticos afetam a percepção do cliente é essencial para orientar decisões de negócio, priorização de vendedores e estratégias comerciais por região.

Dessa forma, o objetivo do presente trabalho é construir um pipeline de dados de ponta a ponta, na nuvem, a partir da base pública da Olist (marketplace brasileiro), respondendo às seguintes perguntas de negócio:

- 1) O tempo de entrega (prazo real vs. prazo estimado) influencia a nota de avaliação dada pelo cliente?
- 2) Quais categorias de produto têm maior taxa de atraso na entrega?
- 3) Quais estados/regiões do Brasil geram maior receita total e maior ticket médio por pedido?
- 4) Existe relação entre o valor do frete e a probabilidade de o cliente dar uma nota baixa?
- 5) Quais vendedores concentram o maior volume de vendas e como é o desempenho de entrega deles?

Estas perguntas guiam a escolha das tabelas, a modelagem dos dados e a análise final. Perguntas eventualmente respondidas apenas de forma parcial são discutidas na seção de Autoavaliação, ao final deste documento.

Detalhamento do projeto

Dados e ferramentas selecionadas

Para responder aos questionamentos propostos, a fonte de dados escolhida foi:

- Brazilian E-Commerce Public Dataset by Olist (Kaggle) — dados reais e anonimizados de aproximadamente 100 mil pedidos realizados entre 2016 e 2018 em múltiplos marketplaces brasileiros integrados pela Olist.

Licença: CC BY-NC-SA 4.0 — uso não comercial permitido, com atribuição, compartilhado sob a mesma licença. Adequada para este trabalho acadêmico.

O conjunto de dados é distribuído em 8 arquivos CSV:

- olist_orders_dataset — pedidos: status, datas de compra, aprovação, envio e entrega.
- olist_order_items_dataset — itens de cada pedido: produto, vendedor, preço e frete.
- olist_order_payments_dataset — forma, parcelas e valor de pagamento.
- olist_order_reviews_dataset — avaliações dos clientes (nota, comentário).
- olist_customers_dataset — localização do cliente (cidade, estado, CEP).
- olist_sellers_dataset — localização do vendedor.

- `olist_products_dataset` — categoria, dimensões e peso do produto.
- `product_category_name_translation` — tradução das categorias (PT → EN).

Nota: o arquivo `olist_geolocation_dataset` (coordenadas por CEP) foi conscientemente excluído da coleta por ser significativamente mais pesado que as demais tabelas e por não ser necessário para responder às 5 perguntas de negócio definidas — as dimensões de cliente e vendedor já obtêm cidade e estado diretamente das tabelas de clientes e vendedores, sem depender de geolocalização.

O projeto foi executado inteiramente na plataforma Databricks, em sua versão Free Edition, dentro do catálogo mvp (Unity Catalog), organizado nos schemas staging (arquivos brutos), bronze, silver e gold, seguindo a Arquitetura Medalhão. Os scripts foram desenvolvidos em notebooks Python (PySpark) e Spark SQL.

Coleta e carregamento de dados

O pipeline foi construído seguindo a Arquitetura Medalhão, com a ingestão e as transformações organizadas em três camadas sucessivas — Bronze, Silver e Gold — descritas nesta ordem a seguir.

Camada Bronze

Nesta camada os dados são carregados exatamente como recebidos da fonte, sem qualquer transformação, servindo como cópia fiel e rastreável da origem.

- Criação do catálogo mvp e dos schemas staging, bronze, silver e gold via Spark SQL.
- Criação de um Volume do Unity Catalog (`mvp.staging.olist`) para receber os arquivos brutos.
- Cópia dos 8 arquivos CSV do GitHub para o Volume, utilizando `dbutils.fs.cp` a partir da URL raw do repositório (notebook MVP-download).
- Leitura de cada CSV com `spark.read (header=True, sep=',', multiLine=True, escape='\"')` e gravação como tabela Delta na camada Bronze com `.saveAsTable('mvp.bronze.<nome_tabela>')` (notebook MVP-Bronze), sem qualquer transformação — preservando os dados exatamente como recebidos.

```
▶ ✓ Yesterday (2s) 1

%sql
use catalog mvp;
use schema staging;

> See performance \(2\)

OK

This result is stored as _sqlidf and can be used in other Python and SQL cells.

▶ ✓ 23 hours ago (2s) 2

%sql
create volume olist

> See performance \(1\)

OK

This result is stored as _sqlidf and can be used in other Python and SQL cells.

▶ ✓ 23 hours ago (11s) 3

base_url = "https://raw.githubusercontent.com/PietroThompson/MVP-Data_engineering/refs/heads/main/"
target = "dbfs:/Volumes/mvp/staging/olist/"

files = [
    "olist_customers_dataset.csv",
    "olist_order_items_dataset.csv",
    "olist_order_payments_dataset.csv",
    "olist_order_reviews_dataset.csv",
    "olist_orders_dataset.csv",
    "olist_products_dataset.csv",
    "olist_sellers_dataset.csv",
    "product_category_name_translation.csv",
]

for f in files:
    dbutils.fs.cp(base_url + f, target + f)
    print("Copied", f)
```

Figura 1 – notebook MVP-download: criação do catálogo mvp, do schema staging e do volume olist, seguida da cópia dos 8 CSVs do GitHub para o Volume.

A célula acima executa três etapas em sequência: seleciona o catálogo e schema de staging, cria o Volume que vai receber os arquivos brutos e, por fim, percorre a lista dos 8 arquivos do dataset Olist copiando cada um do repositório GitHub (via URL raw) para o Volume com dbutils.fs.cp.

```
▶ ✓ 23 hours ago (1s) 1 Python

spark.sql("use catalog mvp")
spark.sql("use schema bronze")

> See performance \(2\)

Dataframe[]

✓ Checked for improvements

▶ ✓ 23 hours ago (23s) 2

volume_path = "dbfs:/Volumes/mvp/staging/olist/"

files = [
    "olist_customers_dataset.csv",
    "olist_order_items_dataset.csv",
    "olist_order_payments_dataset.csv",
    "olist_order_reviews_dataset.csv",
    "olist_orders_dataset.csv",
    "olist_products_dataset.csv",
    "olist_sellers_dataset.csv",
    "product_category_name_translation.csv",
]

for f in files:
    table_name = f.replace("olist_", "").replace("_dataset", "").replace(".csv", "")

    df = (spark.read
        .option("header", True)
        .option("sep", ",")
        .option("multiline", True)
        .option("escape", "")
        .csv(volume_path + f))

    df.write.mode("overwrite").saveAsTable(f"mvp.bronze.{table_name}")
    display(df.limit(5))

> See performance \(16\)

✓ Checked for improvements
```

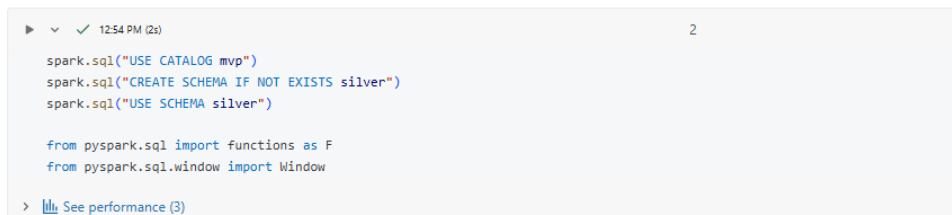
Figura 2 – notebook MVP-Bronze: leitura dos CSVs do Volume e gravação como tabelas Delta na camada Bronze.

Aqui o notebook seleciona o schema bronze e percorre a mesma lista de arquivos: para cada CSV, deriva o nome da tabela (removendo os prefixos/sufixos do nome do arquivo), lê o arquivo com `spark.read` e grava o resultado com `.saveAsTable(f"mvp.bronze.{table_name}")`, sem nenhuma transformação — esta é a essência da camada Bronze.

Camada Silver

Após a ingestão em Bronze, os dados passaram pelos seguintes processos de padronização na camada Silver:

- Remoção de duplicatas: por (`order_id`, `order_item_id`) em itens, por `review_id` em avaliações, mantendo apenas a review mais recente por pedido quando havia mais de uma.
- Tipagem correta: datas convertidas para timestamp; price, freight_value e payment_value convertidos para decimal; contadores para inteiro.
- Enriquecimento: join de products com product_category_name_translation para trazer a categoria já em inglês, evitando repetir esse join em toda consulta futura.
- Regra de acurácia: itens com price ≤ 0 foram descartados por não representarem transações válidas.



```
spark.sql("USE CATALOG mvp")
spark.sql("CREATE SCHEMA IF NOT EXISTS silver")
spark.sql("USE SCHEMA silver")

from pyspark.sql import functions as F
from pyspark.sql.window import Window
```

> [See performance \(3\)](#)

Figura 3 – notebook MVP-Silver: seleção do catálogo/schema silver e importação das funções do PySpark utilizadas nas transformações seguintes.



```
orders_silver = (
    spark.table("mvp.bronze.orders")
    .dropDuplicates(["order_id"])
    .withColumn("order_purchase_timestamp", F.to_timestamp("order_purchase_timestamp"))
    .withColumn("order_approved_at", F.to_timestamp("order_approved_at"))
    .withColumn("order_delivered_carrier_date", F.to_timestamp("order_delivered_carrier_date"))
    .withColumn("order_delivered_customer_date", F.to_timestamp("order_delivered_customer_date"))
    .withColumn("order_estimated_delivery_date", F.to_timestamp("order_estimated_delivery_date"))
)

orders_silver.write.mode("overwrite").saveAsTable("mvp.silver.orders")
display(orders_silver.limit(5))
```

> [See performance \(2\)](#)

Figura 4 – construção de silver.orders: conversão das colunas de data para timestamp e remoção de duplicatas por order_id.

```

▶ 12:54 PM (6s) 6

items_silver = (
    spark.table("mvp.bronze.order_items")
    .dropDuplicates(["order_id", "order_item_id"])
    .withColumn("order_item_id", F.col("order_item_id").cast("int"))
    .withColumn("price", F.col("price").cast("decimal(10,2)"))
    .withColumn("freight_value", F.col("freight_value").cast("decimal(10,2)"))
    .withColumn("shipping_limit_date", F.to_timestamp("shipping_limit_date"))
    .filter(F.col("price") > 0)
)
items_silver.write.mode("overwrite").saveAsTable("mvp.silver.order_items")
display(items_silver.limit(5))

> See performance \(2\)

```

Figura 5 – construção de `silver.order_items`: tipagem de `price` e `freight_value` como decimal e remoção de itens com preço não positivo (filter `price > 0`).

```

▶ 12:54 PM (5s) 8

payments_silver = (
    spark.table("mvp.bronze.order_payments")
    .dropDuplicates(["order_id", "payment_sequential"])
    .withColumn("payment_sequential", F.col("payment_sequential").cast("int"))
    .withColumn("payment_installments", F.col("payment_installments").cast("int"))
    .withColumn("payment_value", F.col("payment_value").cast("decimal(10,2)"))
)
payments_silver.write.mode("overwrite").saveAsTable("mvp.silver.order_payments")
display(payments_silver.limit(5))

> See performance \(2\)

```

Figura 6 – construção de `silver.order_payments`: tipagem das colunas de parcelamento (`payment_installments`) e valor de pagamento (`payment_value`).

```

▶ 12:54 PM (6s) 10

reviews_typed = (
    spark.table("mvp.bronze.order_reviews")
    .withColumn("review_score", F.col("review_score").cast("int"))
    .withColumn("review_creation_date", F.to_timestamp("review_creation_date"))
    .withColumn("review_answer_timestamp", F.to_timestamp("review_answer_timestamp"))
)
w = Window.partitionBy("order_id").orderBy(F.col("review_creation_date").desc())
reviews_silver = (
    reviews_typed.withColumn("rn", F.row_number().over(w))
    .filter("rn = 1")
    .drop("rn")
)
reviews_silver.write.mode("overwrite").saveAsTable("mvp.silver.order_reviews")
display(reviews_silver.limit(5))

> See performance \(2\)

```

Figura 7 – construção de `silver.order_reviews`: mantém apenas a review mais recente por pedido, usando uma Window particionada por `order_id` e ordenada por `review_creation_date`.

```

▶ 12:54 PM (9s) 12

customers_silver = spark.table("mvp.bronze.customers").dropDuplicates(["customer_id"])
customers_silver.write.mode("overwrite").saveAsTable("mvp.silver.customers")

sellers_silver = spark.table("mvp.bronze.sellers").dropDuplicates(["seller_id"])
sellers_silver.write.mode("overwrite").saveAsTable("mvp.silver.sellers")

display(customers_silver.limit(5))

> See performance \(3\)

```

Figura 8 – construção de *silver.customers* e *silver.sellers*: remoção de duplicatas por *customer_id* e *seller_id*, respectivamente.

```
12:54 PM (6s) 14
translation = spark.table("mvp.bronze.product_category_name_translation")

products_silver = (
    spark.table("mvp.bronze.products")
    .dropDuplicates(["product_id"])
    .join(translation, on="product_category_name", how="left")
    .withColumn(
        "product_category_name_english",
        F.coalesce(F.col("product_category_name_english"), F.lit("undefined")),
    )
    .withColumn("product_weight_g", F.col("product_weight_g").cast("int"))
    .withColumn("product_length_cm", F.col("product_length_cm").cast("int"))
    .withColumn("product_height_cm", F.col("product_height_cm").cast("int"))
    .withColumn("product_width_cm", F.col("product_width_cm").cast("int"))
)
products_silver.write.mode("overwrite").saveAsTable("mvp.silver.products")
display(products_silver.limit(5))
> See performance \(2\)
```

Figura 9 – construção de *silver.products*: join com a tradução de categorias e tipagem das dimensões físicas do produto (peso, comprimento, altura, largura).

Camada Gold

Na camada Gold foi construído um esquema estrela: quatro dimensões (*dim_cliente*, *dim_vendedor*, *dim_produto*, *dim_tempo*) e uma tabela fato, *gold.fato_pedidos*, no grão de um item de pedido. Por decisão de modelagem, os atributos descritivos de cidade/estado do cliente e do vendedor, além da categoria do produto, foram trazidos via JOIN diretamente para dentro da fato (denormalização proposital), simplificando as consultas de análise sem abrir mão das dimensões documentadas separadamente para fins de catálogo.

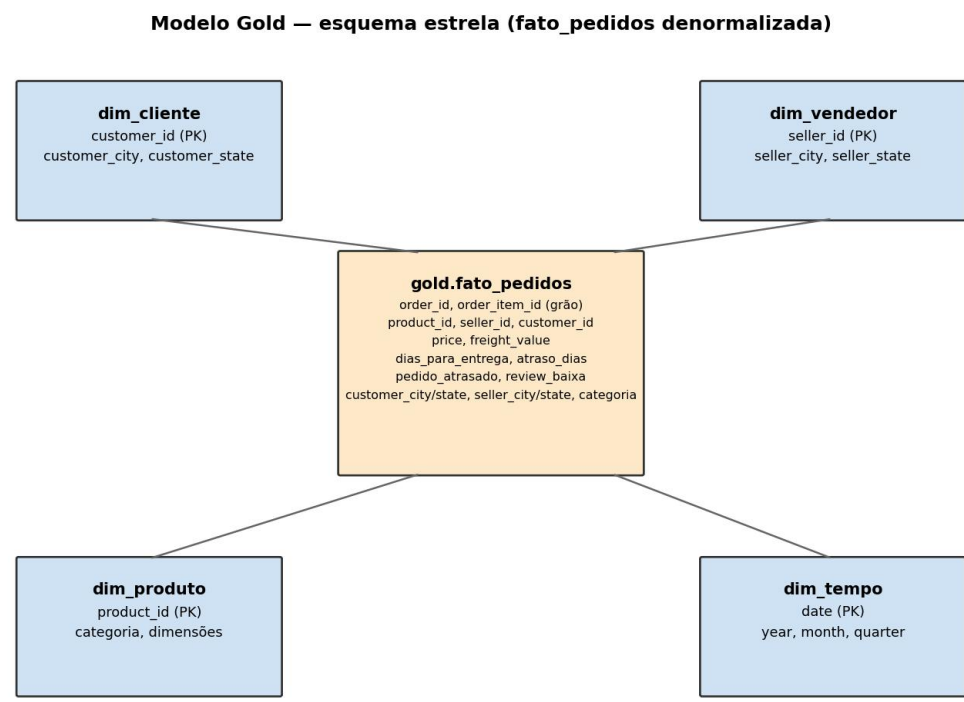


Figura 10 – Modelo Gold em esquema estrela (fato_pedidos denormalizada).

Catálogo de dados

A seguir, o catálogo de dados da tabela principal da camada Gold, gold.fato_pedidos:

Campo	Tipo	Domínio / valores esperados	Linhagem
order_id, order_item_id	string, int	chave composta — grão da tabela	silver.order_items
customer_id / seller_id / product_id	string	chaves estrangeiras das dimensões	silver.order_items
price	decimal(10,2)	> 0 (itens ≤ 0 descartados na Silver)	silver.order_items.price
freight_value	decimal(10,2)	≥ 0	silver.order_items.freight_value
dias_para_entrega	int	≥ 0 (dias entre compra e entrega real)	calculado: datediff
atraso_dias	int	pode ser negativo (entregue antes do prazo)	calculado: datediff
pedido_atrasado	int (flag)	1 se atraso_dias > 0; 0 caso contrário; nulo se sem entrega	calculado
review_score	int	1 a 5 (nulo se sem avaliação)	silver.order_reviews.review_score

Campo	Tipo	Domínio / valores esperados	Linhagem
review_baixa	int (flag)	1 se review_score ≤ 2; 0 caso contrário	calculado
customer_city / customer_state	string	cidade e UF do cliente	silver.customers (join)
seller_city / seller_state	string	cidade e UF do vendedor	silver.sellers (join)
categoria	string	categoria do produto em inglês ('undefined' se sem tradução)	silver.products (join)

Tabela 1 – Catálogo de dados de gold.fato_pedidos.

Além da tabela fato, documentada manualmente acima, a camada Bronze foi documentada de forma automática utilizando a funcionalidade de comentários gerados por Inteligência Artificial do Unity Catalog (AI-generated comments): ao abrir cada tabela no Catalog Explorer do Databricks, é possível clicar em "AI generate" para que o próprio Databricks analise os dados e escreva uma descrição da tabela e de cada coluna. Os comentários abaixo foram exportados diretamente da plataforma e não foram escritos manualmente.

Comentários de tabela (camada Bronze), gerados pela IA do Databricks:

Tabela	Comentário gerado pela IA
customers	The table contains information about customers, including identifiers and their location details such as city and state. This data can be used for demographic analysis, understanding customer distribution, and enhancing marketing strategies based on geographic insights.
order_items	The table contains details about items within customer orders. It includes information such as order IDs, product IDs, seller IDs, prices, and shipping limits. Possible use cases for this data include analyzing sales performance by product or seller, tracking shipping deadlines, and understanding pricing structures.
order_payments	The table captures payment information related to customer orders. It includes details such as the order identifier, payment type, and the value of each payment. This data can be used for analyzing payment trends, understanding customer preferences regarding payment methods, and managing financial reports.
order_reviews	The table contains customer reviews for orders, capturing key details such as review scores, comments, and timestamps for when the reviews were created and answered. This data can be used to assess customer satisfaction, track feedback trends, and evaluate the performance of products or services.

Tabela	Comentário gerado pela IA
orders	The table contains data on customer orders, capturing key details related to each order's lifecycle. It includes information such as the customer associated with each order, the order's status, and timestamps for various stages like purchase, approval, and delivery. This data can be used to track order progress, analyze delivery performance, and understand customer purchasing patterns.
product_category_name_translation	The table includes translations for product category names, capturing both the original category name and its English equivalent. This data can be used for localization purposes, ensuring that product categories are correctly displayed in different languages, and can help in maintaining consistency across various platforms.
products	The table contains detailed information about products available in the inventory. It includes data such as product IDs, categories, dimensions, weight, and descriptions. Potential use cases include analyzing product offerings, optimizing inventory management, and enhancing product listings for better customer engagement.
sellers	The table contains information about sellers, including their identification and location details. This data can be useful for analyzing seller distribution, understanding regional sales trends, and targeting marketing efforts based on geographic regions.

Tabela 2 – Comentários de tabela da camada Bronze, gerados automaticamente pela IA do Unity Catalog (Databricks).

Comentários de coluna (camada Bronze), gerados pela IA do Databricks:

Tabela	Coluna	Tipo	Comentário gerado pela IA
customers	customer_id	STRING	Represents a unique identifier assigned to each customer, which is essential for tracking customer interactions and maintaining accurate records.
customers	customer_unique_id	STRING	A distinct ID that further ensures uniqueness among customers, helpful for data integration and cross-referencing across different systems.
customers	customer_zip_code_prefix	STRING	Indicates the zip code prefix associated with the customer, which can be utilized for geographic segmentation and targeted marketing efforts.
customers	customer_city	STRING	Denotes the city where the customer is located, providing

Tabela	Coluna	Tipo	Comentário gerado pela IA
			insights into regional distribution and potential market reach.
customers	customer_state	STRING	Represents the state of residence for the customer, important for understanding geographic demographics and regional trends.
order_items	order_id	STRING	Identifier for the customer's order linking multiple order items together under a single purchase.
order_items	order_item_id	STRING	Unique identifier for each individual item within an order, distinguishing multiple products in the same order.
order_items	product_id	STRING	Identifier for the specific product included in the order item, useful for product-level analysis.
order_items	seller_id	STRING	Identifier for the seller responsible for fulfilling the order item, allowing tracking of sales performance by seller.
order_items	shipping_limit_date	STRING	The latest date by which the item should be shipped to meet delivery expectations.
order_items	price	STRING	The sale price charged for the product in this order item, used in revenue calculations.
order_items	freight_value	STRING	The cost of shipping the order item, relevant for understanding logistics expenses.
order_payments	order_id	STRING	The unique identifier for the order associated with the payment, allowing for tracing and linking payment details to specific customer orders.
order_payments	payment_sequential	STRING	Indicates the sequence number of the payment for a particular order, which is useful for tracking multiple payments made towards the same order.
order_payments	payment_type	STRING	Describes the method used for the payment, such as credit card,

Tabela	Coluna	Tipo	Comentário gerado pela IA
			debit card, or bank transfer, providing insights into customer payment preferences.
order_payments	payment_installments	STRING	Specifies the number of installments agreed upon for the payment, which helps in understanding customer financing options.
order_payments	payment_value	STRING	Represents the total amount of money paid in the transaction, essential for financial analysis and reporting.
order_reviews	review_id	STRING	Unique identifier assigned to each customer review for precise tracking and reference.
order_reviews	order_id	STRING	Identifier linking the review to the specific order it pertains to, enabling order-specific feedback analysis.
order_reviews	review_score	STRING	Rating given by the customer to indicate their level of satisfaction with the order or product.
order_reviews	review_comment_title	STRING	Brief headline summarizing the customer's main feedback or sentiment in the review.
order_reviews	review_comment_message	STRING	Detailed textual feedback provided by the customer elaborating on their experience or opinion.
order_reviews	review_creation_date	STRING	Timestamp indicating when the customer submitted the review, useful for time-based analysis of feedback.
order_reviews	review_answer_timestamp	STRING	Timestamp marking when the review received a response, helping track responsiveness to customer feedback.
orders	order_id	STRING	A unique identifier for each order placed by a customer, which helps in tracking the specific details pertaining to that order.

Tabela	Coluna	Tipo	Comentário gerado pela IA
orders	customer_id	STRING	Identifies the customer who placed the order, linking the order to their profile for further analysis and tracking.
orders	order_status	STRING	Represents the current status of the order, indicating whether it is pending, approved, delivered, or any other relevant state.
orders	order_purchase_timestamp	STRING	Records the exact date and time when the order was purchased, allowing for tracking of purchasing trends and order lifecycle.
orders	order_approved_at	STRING	Captures the date and time when the order was approved, providing insight into processing times and efficiency.
orders	order_delivered_carrier_date	STRING	Indicates the date when the order was handed over to the delivery carrier, marking a key transition point in the fulfillment process.
orders	order_delivered_customer_date	STRING	Shows the date when the customer actually received the order, which is essential for evaluating delivery performance and customer satisfaction.
orders	order_estimated_delivery_date	STRING	Provides an estimated date of when the order is expected to be delivered to the customer, useful for planning and expectations management.
product_category_name_translation	product_category_name	STRING	Contains the original name of the product category, which may be in a different language depending on the source data.
product_category_name_translation	product_category_name_english	STRING	Represents the English translation of the product category name, ensuring accessibility and understanding for English-speaking users.
products	product_id	STRING	Unique identifier assigned to each product in the inventory, facilitating tracking and management.

Tabela	Coluna	Tipo	Comentário gerado pela IA
products	product_category_name	STRING	Represents the category to which the product belongs, aiding in organization and filter options for customers.
products	product_name_lenght	STRING	Indicates the length of the product name, which can impact browsing and searchability on the platform.
products	product_description_lenght	STRING	Shows the length of the product description, providing insight into the detail level provided to customers.
products	product_photos_qty	STRING	Denotes the quantity of photos available for each product, which can enhance customer engagement and decision-making.
products	product_weight_g	STRING	Represents the weight of the product in grams, useful for shipping calculations and understanding product handling.
products	product_length_cm	STRING	Indicates the length of the product in centimeters, essential for customers assessing fit and suitability.
products	product_height_cm	STRING	Shows the height of the product in centimeters, important for understanding the product's dimensions and space requirements.
products	product_width_cm	STRING	Represents the width of the product in centimeters, aiding customers in determining if the product will fit their needs.
sellers	seller_id	STRING	A unique identifier assigned to each seller, which can be used to track their performance and sales activities.
sellers	seller_zip_code_prefix	STRING	Represents the initial digits of the seller's postal code, useful for geographic analysis and regional marketing strategies.
sellers	seller_city	STRING	Indicates the city where the seller is located, providing insights into

Tabela	Coluna	Tipo	Comentário gerado pela IA
			regional market dynamics and seller distribution.
sellers	seller_state	STRING	Denotes the state in which the seller operates, which can assist in understanding state-specific sales trends and regulatory considerations.

Tabela 3 – Comentários de coluna da camada Bronze, gerados automaticamente pela IA do Unity Catalog (Databricks). Todas as colunas aparecem como STRING nesta camada porque a ingestão Bronze não realiza inferência ou conversão de tipos — a tipagem correta é aplicada apenas na camada Silver (ver Figuras 4 a 9).

As dimensões dim_cliente, dim_vendedor, dim_produto e dim_tempo, na camada Gold, seguem o mesmo padrão de documentação e podem ser expandidas da mesma forma diretamente no Unity Catalog.

Análise de dados

Qualidade de dados

A checagem de qualidade foi realizada com apoio da ferramenta nativa Data Profile do Databricks (disponível ao expandir o resultado de qualquer display()) e de validações pontuais em PySpark. Os principais achados:

- **Completeness:** cerca de 2,98% dos pedidos não possuem order_delivered_customer_date preenchida — em geral pedidos cancelados, extraviados ou ainda em trânsito no momento da extração da base. Esses casos não foram removidos, mas ficam automaticamente fora das métricas de atraso (categorizados como "sem entrega").
- **Consistency of status:** identificados 8 valores distintos de order_status (delivered, shipped, canceled, unavailable, invoiced, processing, created, approved), com "delivered" representando ~97% dos pedidos.
- **Uniqueness:** duplicatas exatas encontradas em order_items (por order_id + order_item_id) e em order_reviews (por review_id); ambas tratadas na camada Silver. Pedidos com mais de uma review (reenviada pelo cliente) tiveram apenas a mais recente mantida.
- **Accuracy:** itens com price ≤ 0 foram descartados da camada Silver como regra de salvaguarda (nenhum caso relevante encontrado na prática).
- **Outliers:** não foi aplicado tratamento estatístico de outliers de valor nesta versão do MVP — registrado como ponto de melhoria.

Solução do problema proposto

A partir de gold.fato_pedidos, já denormalizada com os atributos de cliente, vendedor e produto, foi possível responder diretamente às 5 perguntas de negócio utilizando a API de DataFrame do PySpark (.groupBy().agg()), com gráficos gerados automaticamente via matplotlib.

1) O atraso na entrega influencia a nota de avaliação?

```
q1_df = (
    fato
    .filter(F.col("review_score").isNotNull())
    .withColumn(
        "faixa_atraso",
        F.when(F.col("atraso_dias").isNull(), "sem entrega")
        .when(F.col("atraso_dias") <= -7, "muito adiantado (7+ dias)")
        .when(F.col("atraso_dias") < 0, "adiantado")
        .when(F.col("atraso_dias") == 0, "no prazo")
        .when(F.col("atraso_dias") <= 7, "atraso leve (1-7 dias)")
        .otherwise("atraso grave (8+ dias)"),
    )
    .groupBy("faixa_atraso")
    .agg(
        F.round(F.avg("review_score"), 2).alias("nota_media"),
        F.countDistinct("order_id").alias("pedidos"),
    )
    .orderBy(F.col("nota_media").desc())
)

q1_pdf = q1_df.toPandas()
display(q1_df)
plot_bar(q1_pdf, "faixa_atraso", "nota_media",
         "Nota média de avaliação por faixa de atraso", "Nota média (1-5)")
```

Figura 11 – código PySpark utilizado para responder à pergunta 1 (notebook MVP-Analise).

O código classifica cada pedido em uma faixa de atraso (usando `F.when` encadeado sobre a coluna `atraso_dias`) e, em seguida, agrupa por essa faixa com `groupBy().agg()` para calcular a nota média de avaliação e o número de pedidos em cada uma.

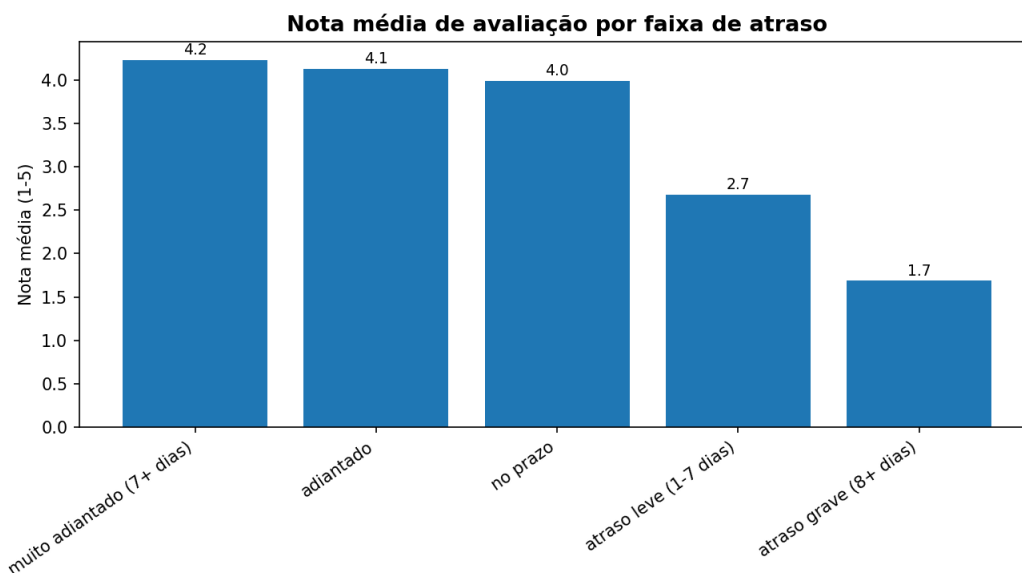


Figura 12 – Nota média de avaliação por faixa de atraso

O efeito é bastante acentuado: a nota média cai de 4,23 (pedidos muito adiantados, 7 ou mais dias) para apenas 1,69 (atrasos de 8 dias ou mais) — uma queda de mais de 2,5 pontos na escala de 1 a 5. Olhando para a distribuição completa das notas por faixa, o padrão fica ainda mais claro: entre os pedidos muito adiantados, 81,0% recebem nota 4 ou 5 e apenas 11,2% recebem nota 1 ou 2; essa proporção se inverte

quase que completamente na faixa de atraso grave, onde 79,5% dos pedidos recebem nota 1 ou 2 e só 11,7% recebem nota 4 ou 5 — uma inversão quase espelhada.

Chama atenção também o tamanho de cada grupo: 79% dos pedidos avaliados chegaram com 7 dias ou mais de antecedência (a grande maioria da base), enquanto apenas 2,9% (2.766 pedidos) caem na faixa de atraso grave. Do ponto de vista de negócio, isso é uma boa notícia operacional — a esmagadora maioria das entregas está adiantada —, mas também reforça que mesmo uma fatia pequena de pedidos malsucedidos tem poder desproporcional de gerar avaliações negativas e prejudicar a reputação do marketplace, o que torna a eliminação de atrasos graves uma prioridade de alto retorno mesmo afetando relativamente poucos pedidos.

2) Quais categorias de produto têm maior taxa de atraso na entrega?



```
q2_df = (
    fato
    .groupBy("categoria")
    .agg(
        F.round(F.avg("pedido_atrasado") * 100, 1).alias("pct_atrasado"),
        F.countDistinct("order_id").alias("pedidos"),
    )
    .filter(F.col("pedidos") >= 30)
    .orderBy(F.col("pct_atrasado").desc())
    .limit(10)
)

q2_pdf = q2_df.toPandas()
display(q2_df)
plot_bar(q2_pdf, "categoria", "pct_atrasado",
         "Top 10 categorias por % de pedidos atrasados (mín. 30 pedidos)",
         "% atrasado", value_fmt="{:.1f}%")
```

Figura 13 – código PySpark utilizado para responder à pergunta 2 (notebook MVP-Analise).

O código agrupa os itens por categoria, calcula o percentual médio de atraso (média da coluna `pedido_atrasado`) e o número de pedidos distintos, filtrando apenas categorias com pelo menos 30 pedidos para evitar que categorias muito pequenas distorçam o ranking.

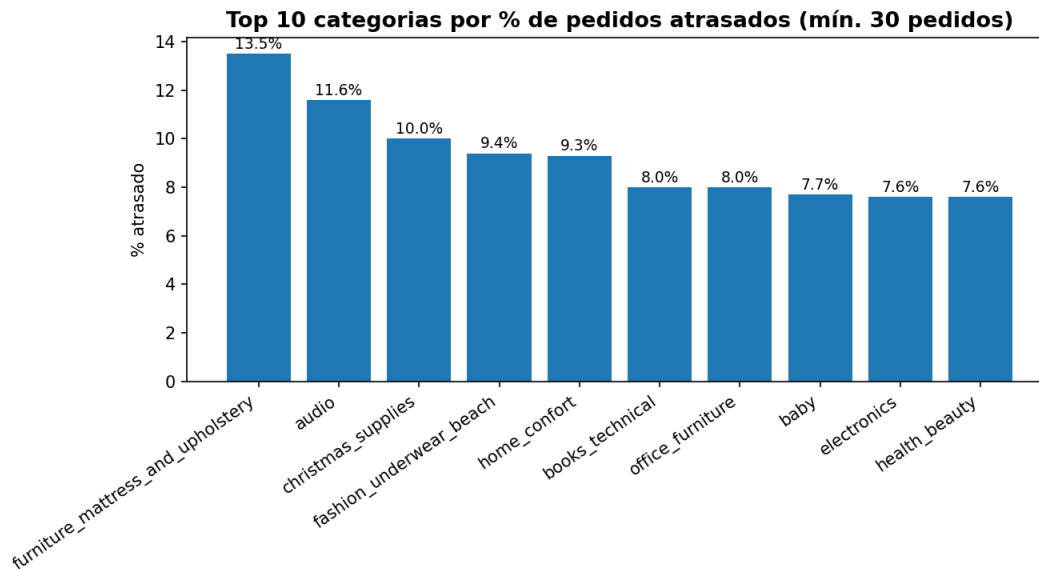


Figura 14 – Top 10 categorias por % de pedidos atrasados

furniture_mattress_and_upholstery (colchões e estofados) lidera com 13,5% de pedidos atrasados — mais do que o dobro da taxa média de atraso da base como um todo (6,6%) —, plausivelmente por serem itens volumosos e mais difíceis de transportar. É importante notar, porém, que essa categoria tem uma base pequena (38 pedidos), o que a torna estatisticamente mais sensível a variações pontuais do que categorias maiores.

Já health_beauty chama atenção por ser, de longe, a categoria de maior volume entre as dez piores (8.836 pedidos — mais de 20 vezes o volume de furniture_mattress_and_upholstery), mesmo com uma taxa de atraso bem mais moderada (7,6%). Em termos absolutos, isso significa muito mais clientes impactados por atrasos nessa categoria do que em qualquer outra do ranking, o que a torna prioritária para ações de melhoria logística mesmo não sendo a pior taxa percentual. Vale registrar também que categorias como audio (11,6% de atraso, 350 pedidos) e christmas_supplies (10,0%, 128 pedidos) — esta última provavelmente concentrada em um pico sazonal de dezembro — reforçam a hipótese de que sazonalidade e volume/dimensão física do produto são fatores relevantes para o risco de atraso.

3) Quais estados geram maior receita total e maior ticket médio por pedido?

```

05:08 PM (2s) 10

q3_df = (
    fato
    .groupBy("customer_state")
    .agg(
        F.round(F.sum("price"), 0).alias("receita_total"),
        F.countDistinct("order_id").alias("pedidos"),
    )
    .withColumn("ticket_medio", F.round(F.col("receita_total") / F.col("pedidos"), 2))
    .orderBy(F.col("receita_total").desc())
    .limit(10)
)

q3_pdf = q3_df.toPandas()
display(q3_df)
plot_bar(q3_pdf, "customer_state", "receita_total",
         "Top 10 estados por receita total", "Receita (R$)", value_fmt="{:,.0f}")
> See performance \(2\)

```

Figura 15 – código PySpark para a receita total por estado (notebook MVP-Analise).

O código agrupa os itens por `customer_state` e calcula a receita total (soma de `price`) e o número de pedidos distintos, ordenando pelos 10 estados de maior receita.

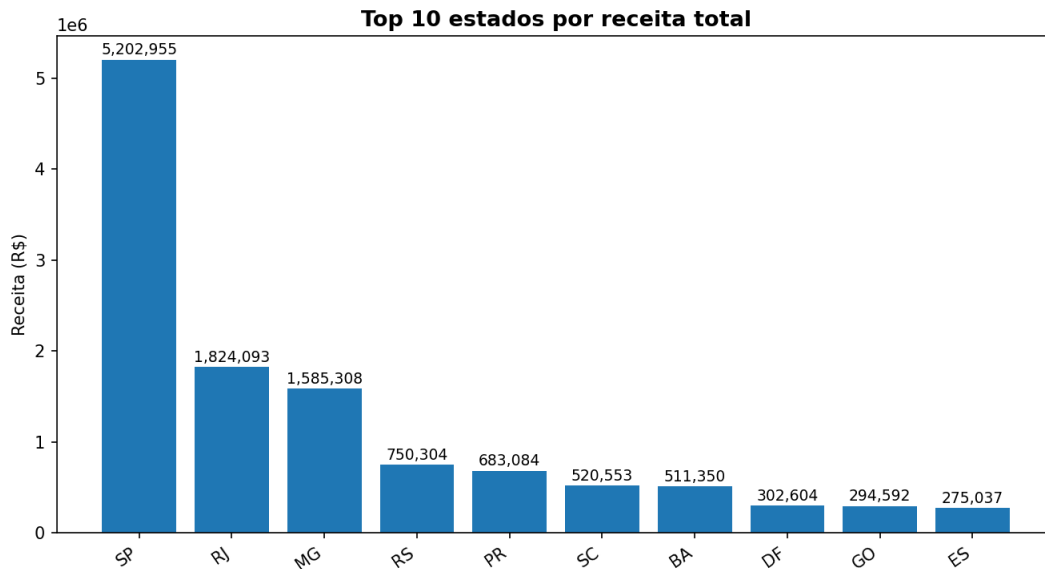


Figura 16 – Top 10 estados por receita total

São Paulo concentra sozinho cerca de 38,3% de toda a receita da base (R\$ 5,2 milhões de um total nacional de R\$ 13,6 milhões), seguido de longe por Rio de Janeiro (13,4%) e Minas Gerais (11,7%) — resultado esperado dada a concentração populacional e de vendedores no Sudeste. Juntos, os quatro maiores estados (SP, RJ, MG e RS) já respondem por 68,9% de toda a receita do marketplace, evidenciando uma concentração geográfica bastante acentuada e reforçando a região Sudeste/Sul como prioridade natural para qualquer estratégia comercial de curto prazo.

```
q3_ticket_df = (
    fato
    .groupBy("customer_state")
    .agg(
        F.round(F.sum("price") / F.countDistinct("order_id"), 2).alias("ticket_medio"),
        F.countDistinct("order_id").alias("pedidos"),
    )
    .filter(F.col("pedidos") >= 100)
    .orderBy(F.col("ticket_medio").desc())
    .limit(10)
)

q3_ticket_pdf = q3_ticket_df.toPandas()
display(q3_ticket_df)
plot_bar(q3_ticket_pdf, "customer_state", "ticket_medio",
         "Top 10 estados por ticket médio (mín. 100 pedidos)",
         "Ticket médio (R$)", value_fmt="R$ {:.0f}")
```

Figura 17 – código PySpark para o ticket médio nacional por estado (notebook MVP-Analise).

Esta é uma consulta separada da anterior: em vez de reordenar apenas os 10 estados de maior receita, ela calcula o ticket médio (receita total dividida pelo número de pedidos distintos) para todos os estados, filtra apenas os que têm pelo menos 100 pedidos — para evitar que estados com poucos pedidos distorçam o ranking por acaso — e então ordena pelos 10 maiores ticket médios.

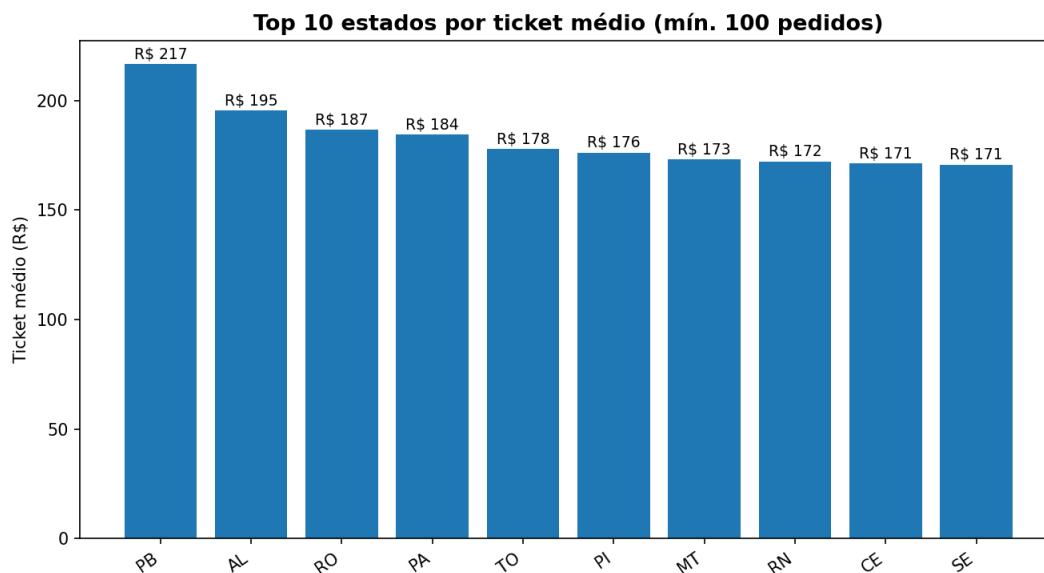


Figura 18 – Top 10 estados por ticket médio (mín. 100 pedidos)

Por outro lado, os estados com maior ticket médio por pedido são majoritariamente do Norte e Nordeste (PB, AL, RO, PA...), com valores entre R\$ 170 e R\$ 217 — de 24% a 57% acima do ticket médio nacional, de R\$ 137,75. Isso sugere que, apesar do menor volume de pedidos, o valor médio das compras nessas regiões é mais alto — possivelmente pelo maior custo de frete embutido no preço final (dada a maior distância dos centros de distribuição) ou pela menor densidade de vendedores locais, o que reduz a concorrência de preço. Do ponto de vista estratégico, isso aponta para dois perfis de mercado distintos dentro da mesma base: um mercado de alto volume e ticket padrão no Sudeste/Sul, e um mercado de menor volume mas ticket premiado no Norte/Nordeste, que pode justificar estratégias comerciais diferenciadas por região.

4) Existe relação entre o valor do frete e a probabilidade de o cliente dar uma nota baixa?

```

05:08 PM (2s) 14

base_q4 = fato.filter((F.col("review_score").isNotNull()) & (F.col("price") > 0))
janela_frete = Window.orderBy(F.col("freight_value") / F.col("price"))

q4_df = (
    base_q4
    .withColumn("quartil_frete_preco", F.ntile(4).over(janela_frete))
    .groupBy("quartil_frete_preco")
    .agg(
        F.round(F.avg("review_baixa") * 100, 1).alias("pct_notas_baixa"),
        F.count("*").alias("itens"),
    )
    .orderBy("quartil_frete_preco")
)

q4_pdf = q4_df.toPandas()
q4_pdf["quartil_frete_preco"] = q4_pdf["quartil_frete_preco"].map(
    {1: "Q1 (menor)", 2: "Q2", 3: "Q3", 4: "Q4 (maior)"}
)

display(q4_df)
plot_bar(q4_pdf, "quartil_frete_preco", "pct_notas_baixa",
        "% de notas baixas (≤2) por quartil de (frete / preço)",
        "% notas baixas", value_fmt="{:.1f}%", color="#d62728")
> See performance \(2\)

```

Figura 19 – código PySpark utilizado para responder à pergunta 4 (notebook MVP-Analise).

O código usa uma Window ordenada pela razão frete/preço (freight_value / price) e a função F.ntile(4) para dividir os itens em quatro quartis de igual tamanho, calculando em seguida o percentual de notas baixas (review_baixa) em cada quartil.

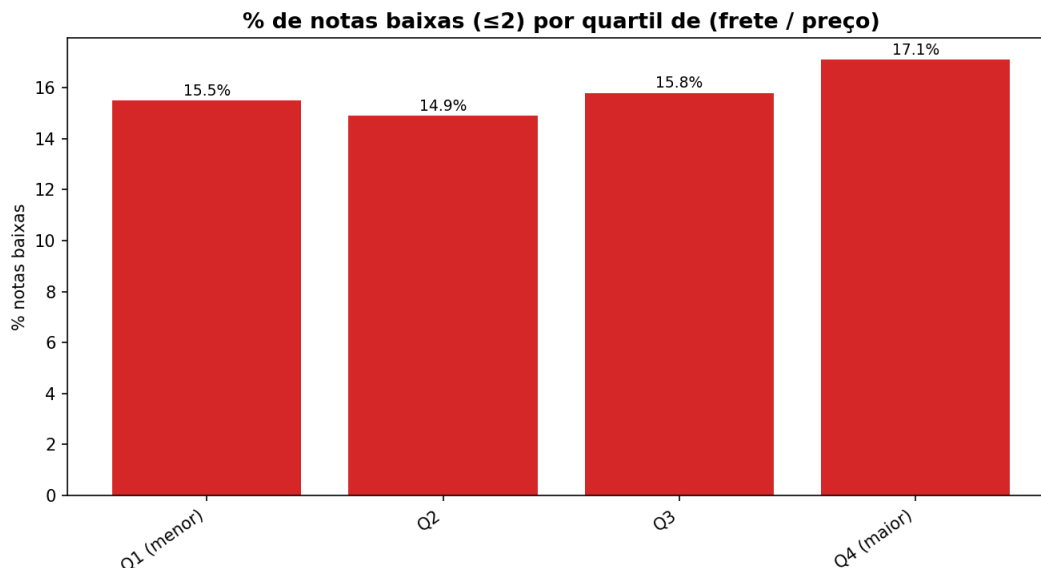


Figura 20 – % de notas baixas (≤2) por quartil de (frete / preço)

Existe uma relação positiva, porém mais discreta que a do atraso: pedidos no quartil de frete proporcionalmente mais caro (Q4) apresentam 17,1% de notas baixas, contra 14,9% no quartil mais barato (Q2) — uma diferença de aproximadamente 2 pontos percentuais. Para dimensionar o que separa esses quartis: no Q1 (mais barato) o frete representa, em média, apenas 8,5% do preço do produto, enquanto no Q4 (mais caro) essa proporção sobe para 71,9% — ou seja, no quartil mais crítico

o frete chega a representar quase três quartos do valor do próprio item, e em muitos casos supera o preço do produto. Essa assimetria ajuda a explicar o efeito: o cliente provavelmente não reage ao valor absoluto do frete, mas à sensação de desproporção entre o que pagou pelo produto e o que pagou para recebê-lo.

Ainda assim, a diferença de 2 pontos percentuais entre os extremos é bem menor do que a variação de mais de 65 pontos percentuais observada entre as faixas de atraso na pergunta 1. Isso indica que, embora o frete desproporcional seja um fator de insatisfação real e mensurável, o cumprimento do prazo de entrega continua sendo, com folga, o principal determinante da experiência do cliente neste marketplace.

5) Quais vendedores concentram o maior volume de vendas e como é o desempenho de entrega deles?



```
q5_df = (
    fato
    .groupBy("seller_id")
    .agg(
        F.round(F.sum("price"), 0).alias("receita_total"),
        F.countDistinct("order_id").alias("pedidos"),
        F.round(F.avg("pedido_atrasado") * 100, 1).alias("pct_atrasado"),
    )
    .orderBy(F.col("receita_total").desc())
    .limit(10)
)

q5_pdf = q5_df.toPandas()
q5_pdf["seller_curto"] = q5_pdf["seller_id"].str[:8]

display(q5_df)
plot_bar(q5_pdf, "seller_curto", "receita_total",
         "Top 10 vendedores por receita total", "Receita (R$)", value_fmt="{:,.0f}")
```

Figura 21 – código PySpark para o ranking de vendedores por receita (notebook MVP-Analise).

O código agrupa os itens por seller_id e calcula a receita total, o número de pedidos distintos e o percentual médio de atraso de cada vendedor, ordenando pelos 10 maiores em receita.

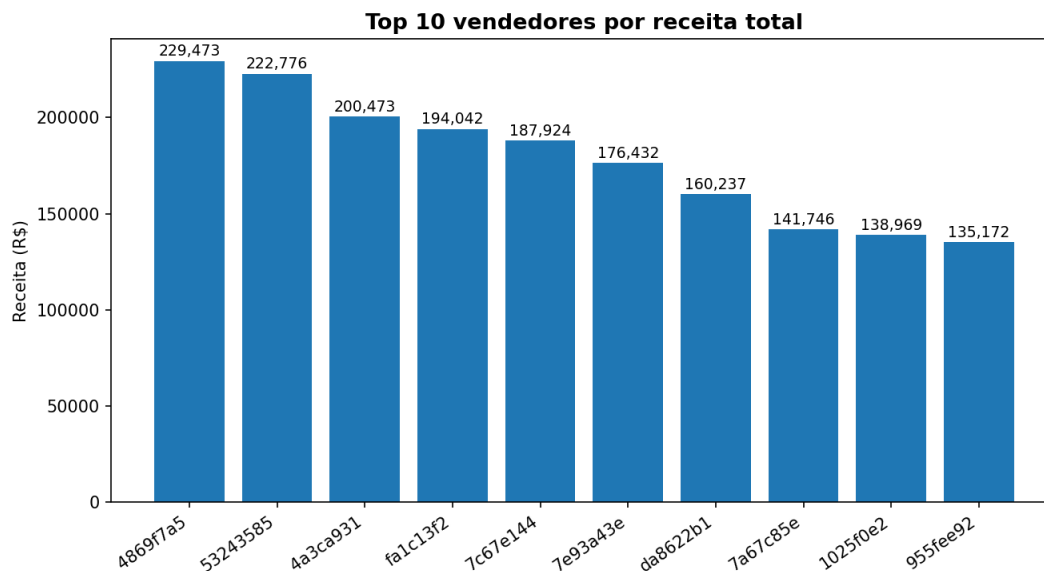


Figura 22 – Top 10 vendedores por receita total

```

05:08 PM (<1s) 18
plot_bar(q5_pdf, "seller_curto", "pct_atrasado",
         "% de pedidos atrasados – mesmos top 10 vendedores por receita",
         "% atrasado", value_fmt="{:.1f}%", color="#d62728")

```

Figura 23 – código do segundo gráfico da pergunta 5, reaproveitando o mesmo DataFrame (q5_pdf).

Como o percentual de atraso de cada vendedor já havia sido calculado na consulta anterior, esta célula apenas gera um segundo gráfico a partir do mesmo q5_pdf, agora destacando pct_atrasado em vez de receita_total, para comparar os dois indicadores lado a lado.

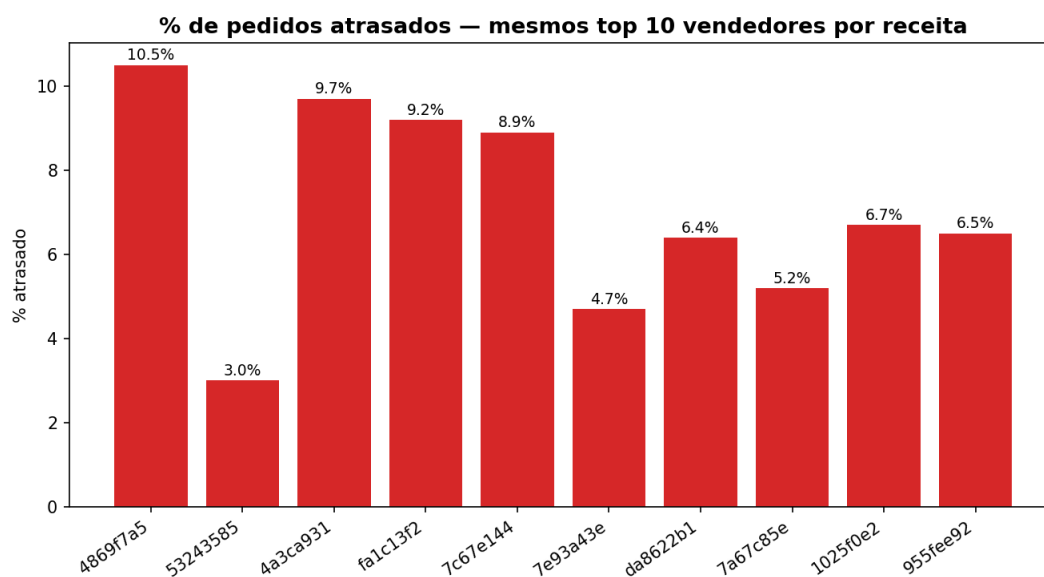


Figura 24 – % de pedidos atrasados dos mesmos top 10 vendedores

A base conta com 3.095 vendedores distintos, e mesmo assim os 10 maiores concentram apenas 13,1% da receita total — um indício de que este é um mercado bastante pulverizado (cauda longa), sem grande dependência de poucos vendedores dominantes, o que reduz o risco de concentração para o marketplace.

Ao cruzar receita com desempenho de entrega, nota-se que o vendedor líder em receita (R\$ 229 mil) tem 10,5% de taxa de atraso — a pior entre os 10 maiores e quase 4 pontos percentuais acima da média geral de atraso da base (6,6%) —, enquanto o 2º colocado em receita (R\$ 223 mil) apresenta apenas 3,0% de atraso, menos da metade da média geral e o melhor resultado da lista. Isso demonstra que alto volume de vendas não garante, por si só, uma boa operação logística: dois vendedores com receitas praticamente equivalentes podem ter desempenhos de entrega radicalmente diferentes. Do ponto de vista comercial, isso sugere que a priorização de vendedores (por exemplo, para destaque na busca ou campanhas promocionais) deveria combinar volume de vendas com indicadores de desempenho logístico, e não apenas receita.

Em conjunto, os resultados confirmam a hipótese inicial de que o desempenho logístico — sobretudo o cumprimento do prazo de entrega — é o principal determinante da satisfação do cliente neste marketplace, superando em importância o valor do frete isoladamente.

Considerações finais

Pontos de melhoria

- Aplicar tratamento estatístico de outliers de valor (price, freight_value) e de tempo (dias_para_entrega), hoje apenas identificados na etapa de Qualidade de Dados mas não tratados quantitativamente.
- Cruzar com dados socioeconômicos externos por estado/região para enriquecer a análise de ticket médio e explicar melhor as diferenças regionais encontradas na pergunta 3.

Autoavaliação

Acredito que este projeto me deu a oportunidade de entender, na prática, a importância de um pipeline de dados para realizar boas análises no futuro. Ao construir cada camada, da Bronze à Gold, vi que as conclusões só são confiáveis quando se conhece o que foi limpo, transformado e descartado ao longo do caminho, e que a análise é apenas a etapa final desse processo.

Quanto aos objetivos, considero que consegui responder às cinco perguntas de negócio definidas, cada uma com análise e gráfico. O resultado mais claro foi o efeito do atraso sobre a satisfação: a nota média cai de 4,23 para 1,69 nos pedidos com atraso de oito dias ou mais, enquanto o frete mostrou uma relação bem mais discreta. Reconheço, porém, que os resultados são associações e não relações de causa e efeito, e que algumas explicações, como o ticket médio mais alto no Norte e Nordeste, são hipóteses que não cheguei a testar.